

P.PORTO

Frameworks do lado do cliente

Programação em Ambiente Web

Índice

Introdução

Typescript

Introdução Angular

Instalação Angular

Demo Angular

Referências

Introdução

No desenvolvimento moderno de aplicações web é frequente o uso de bibliotecas do lado do cliente ou JavaScript frameworks

Este tipo de frameworks tem estado cada vez mais em evidência no mundo da web desde o aparecimento da *framework* Angular

Introdução

Algumas das frameworks mais populares incluem:

- Angular - <https://angular.io/>
- React - <https://reactjs.org/>
- Meteor - <https://www.meteor.com/>
- Vue - <https://vuejs.org/>
- ... (outros)

A escolha depende sempre das características da *framework* e do projeto em questão

Introdução

Como referido anteriormente Angular é parte integrante da MEAN Stack para o desenvolvimento de aplicações web:

- MongoDB
- Express
- Angular
- Node

Existem claro versões alternativas com outras frameworks, mas iremos-mos focar na *framework* Angular

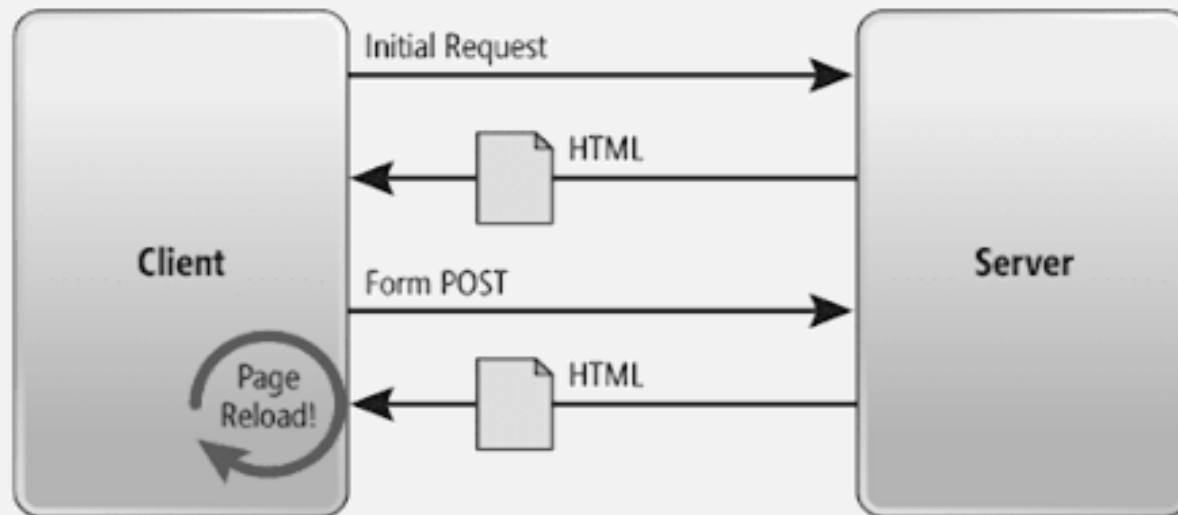
Introdução

Frameworks do lado do cliente como Angular ou React permitem o desenvolvimento de aplicações web baseadas no conceito Single Page Application (SPA)

É também uma boa prática implementar o padrão MVC no desenho das suas aplicações em *frameworks* do lado do cliente

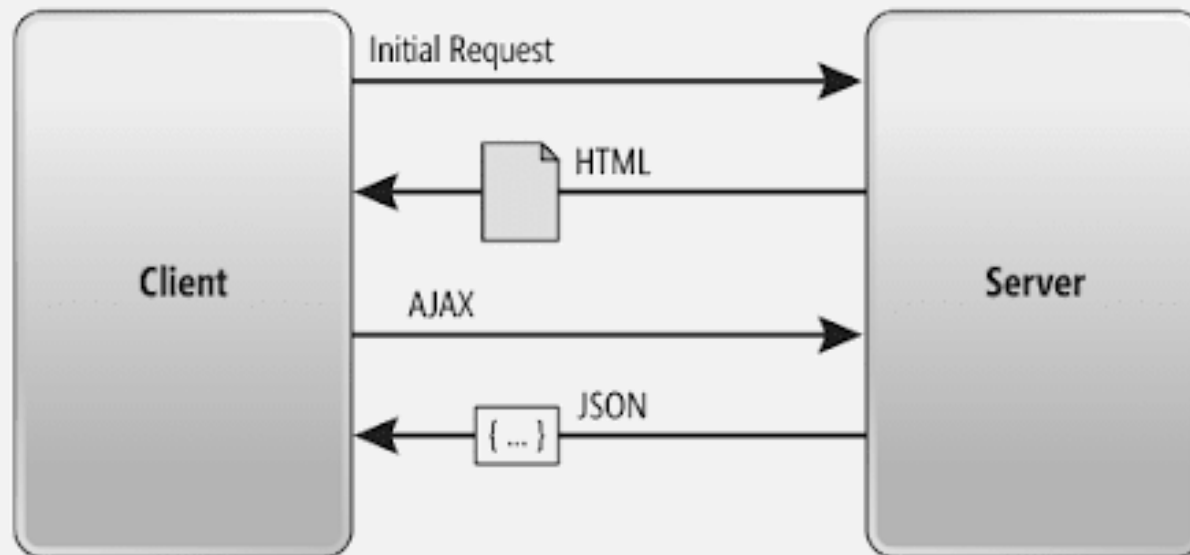
Introdução

MPA – Multiple Page Applications



Introdução

SPA – Single Page Applications



Introdução

As frameworks do lado do cliente usam o conceito app Shell no desenho das suas aplicações web

Neste conceito a estrutura da aplicação é toda descarregada a partir do servidor, usando serviços e pedidos ao servidor subsequentes apenas para atualizar dados

Introdução

Estas frameworks podem usar supersets da linguagem JavaScript que compilam os ficheiros para JavaScript usado pelos browsers:

- Angular usa TypeScript
- React.js usa JSX

Outras frameworks usam diretamente JavaScript

Introdução

Geralmente as frameworks do lado do cliente usam serviços e apis para interagir com recursos da web

É esperado que depois de carregada a página todo o trabalho seja feito do lado cliente e não de um servidor

Funcionam bem em conjunto com serviços e API Rest

Websockets são também uma opção

TypeScript

O TypeScript é uma linguagem compilada fortemente orientada a objetos

Foi projetado por Anders Hejlsberg (designer de C #) na Microsoft

O TypeScript contém não só uma linguagem de programação como um conjunto de ferramentas

TypeScript

O desenvolvimento do NodeJS permitiu ver o JavaScript como uma tecnologia emergente do lado do servidor

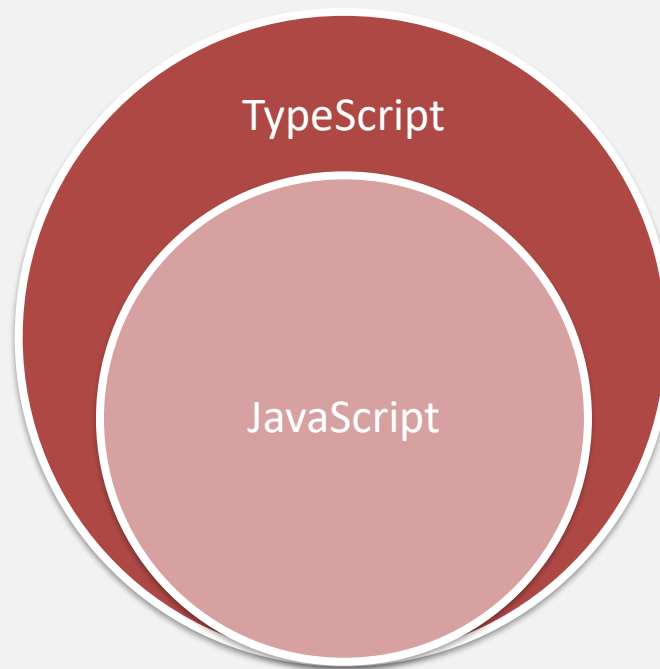
No entanto, à medida que o código JavaScript cresce, ele tende a ficar mais confuso, dificultando a manutenção e a reutilização do código

Além disso, faltava em

- recursos da Orientação a Objetos
- verificação de tipos de variáveis
- verificações de erro em tempo de compilação

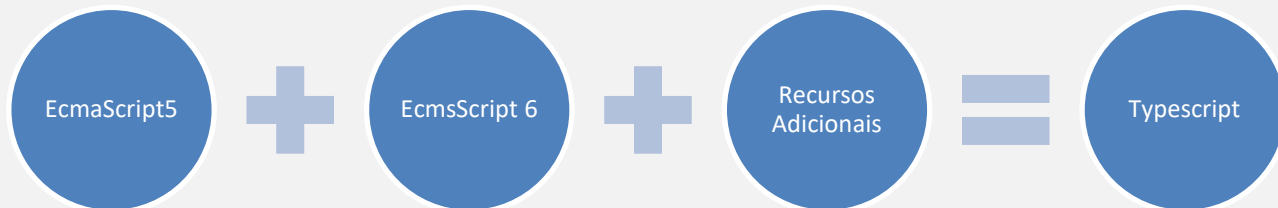
TypeScript

TypeScript é um superset de JavaScript



TypeScript

TypeScript é JavaScript, com alguns recursos adicionais



TypeScript

Todo o código desenvolvido pode ser compilado para JavaScript

TypeScript adota a notação de programação orientada a objectos de javascript e adiciona tipos nas variáveis

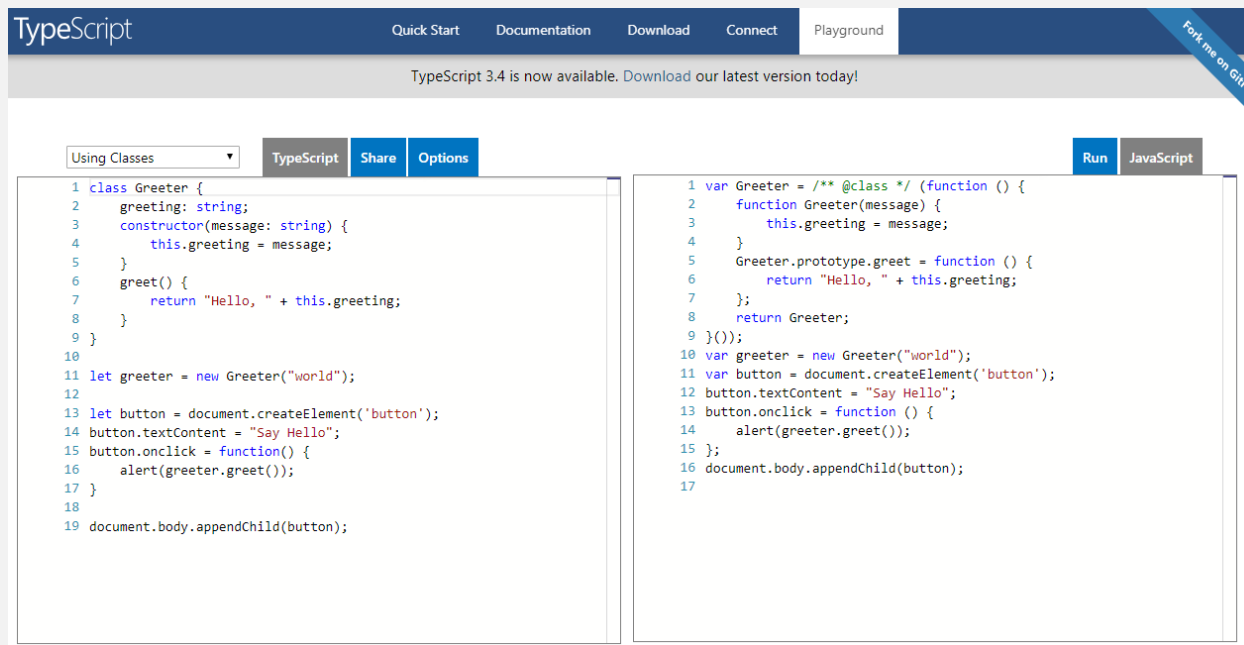
Podem ser usadas bibliotecas de JavaScript convencionais na linguagem

Os ficheiros escritos em typescript terminam em *.ts

TypeScript

Podem experimentar a linguagem e os efeitos da compilação para JavaScript em:

- <https://www.typescriptlang.org/play/>



The screenshot displays the TypeScript Playground interface. At the top, there's a navigation bar with links for Quick Start, Documentation, Download, Connect, and Playground. A banner below the navigation bar states "TypeScript 3.4 is now available. Download our latest version today!". On the right side of the banner, there's a "Fork me on GitHub" button. Below the banner, there's a dropdown menu set to "Using Classes" and buttons for "TypeScript", "Share", and "Options". The main area is split into two panels. The left panel shows the TypeScript code, and the right panel shows the compiled JavaScript code. The "Run" and "JavaScript" buttons are located at the top right of the right panel.

```
1 class Greeter {
2   greeting: string;
3   constructor(message: string) {
4     this.greeting = message;
5   }
6   greet() {
7     return "Hello, " + this.greeting;
8   }
9 }
10
11 let greeter = new Greeter("world");
12
13 let button = document.createElement('button');
14 button.textContent = "Say Hello";
15 button.onclick = function() {
16   alert(greeter.greet());
17 }
18
19 document.body.appendChild(button);
```

```
1 var Greeter = /** @class */ (function () {
2   function Greeter(message) {
3     this.greeting = message;
4   }
5   Greeter.prototype.greet = function () {
6     return "Hello, " + this.greeting;
7   };
8   return Greeter;
9 }());
10 var greeter = new Greeter("world");
11 var button = document.createElement('button');
12 button.textContent = "Say Hello";
13 button.onclick = function () {
14   alert(greeter.greet());
15 };
16 document.body.appendChild(button);
17
```

TypeScript

Um programa tipo em TypeScript faz uso de:

- Módulos
- Funções
- Variáveis
- Expressões, estruturas de controlo e declarações
- Comentários

TypeScript

Palavras Reservadas na linguagem

break	as	any	switch
case	if	throw	else
var	number	string	get
module	type	instanceof	typeof
public	private	enum	export
finally	for	while	void
null	super	this	new
in	return	true	false
any	extends	static	let
package	implements	interface	function
new	try	yield	const
continue	do	catch	

TypeScript

Tipos presentes na linguagem

Data type	Keyword	Description
Number	number	Double precision 64-bit floating point values. It can be used to represent both, integers and fractions.
String	string	Represents a sequence of Unicode characters
Boolean	boolean	Represents logical values, true and false
Void	void	Used on function return types to represent non-returning functions
Null	null	Represents an intentional absence of an object value.
Undefined	undefined	Denotes value given to all uninitialized variables

TypeScript

Declaração de tipos nas variáveis

- `var name:string = "John";`
- `var score1:number = 50;`

Usa os operadores comuns em JavaScript

Usa os controlos condicionais “if” “switch” “do” “while”
tal como JavaScript

TypeScript

Declaração de tipos de retorna e das variáveis na declaração de funções

- function function_name (param1[:type], param2[:type], param3[:type])

```
var res = function(a:number,b:number) {  
    return a*b;  
};  
console.log(res(12,2))
```

TypeScript

Estrutura de programação orientada a objetos

```
class Car {  
  //field  
  engine:string;  
  
  //constructor  
  constructor(engine:string) {  
    this.engine = engine  
  }  
  
  //function  
  disp():void {  
    console.log("Engine is : "+this.engine)  
  }  
}
```

TypeScript

Introdução do tipo any para evitar a deteção de erros e poder declarar uma variável de qualquer tipo

Evita a deteção de erros em tempo de compilação

Necessário cuidado com o seu uso

TypeScript

Exercicio

```
function greeter(person: string) {  
    return "Hello, " + person;  
}  
  
let user = [0, 1, 2];  
  
document.body.innerHTML = greeter(user);
```

TypeScript

Podemos instalar a linguagem TypeScript a partir do package manager npm com:

- `npm install -g typescript`

Escrevemos programas em TypeScript num ficheiro de texto normal com extensão *.ts

TypeScript

Podemos compilar o ficheiro com o comando:

- `tsc file.ts`

Podemos depois executar o ficheiro javascript gerado com node com o comando:

- `node file.js`

TypeScript

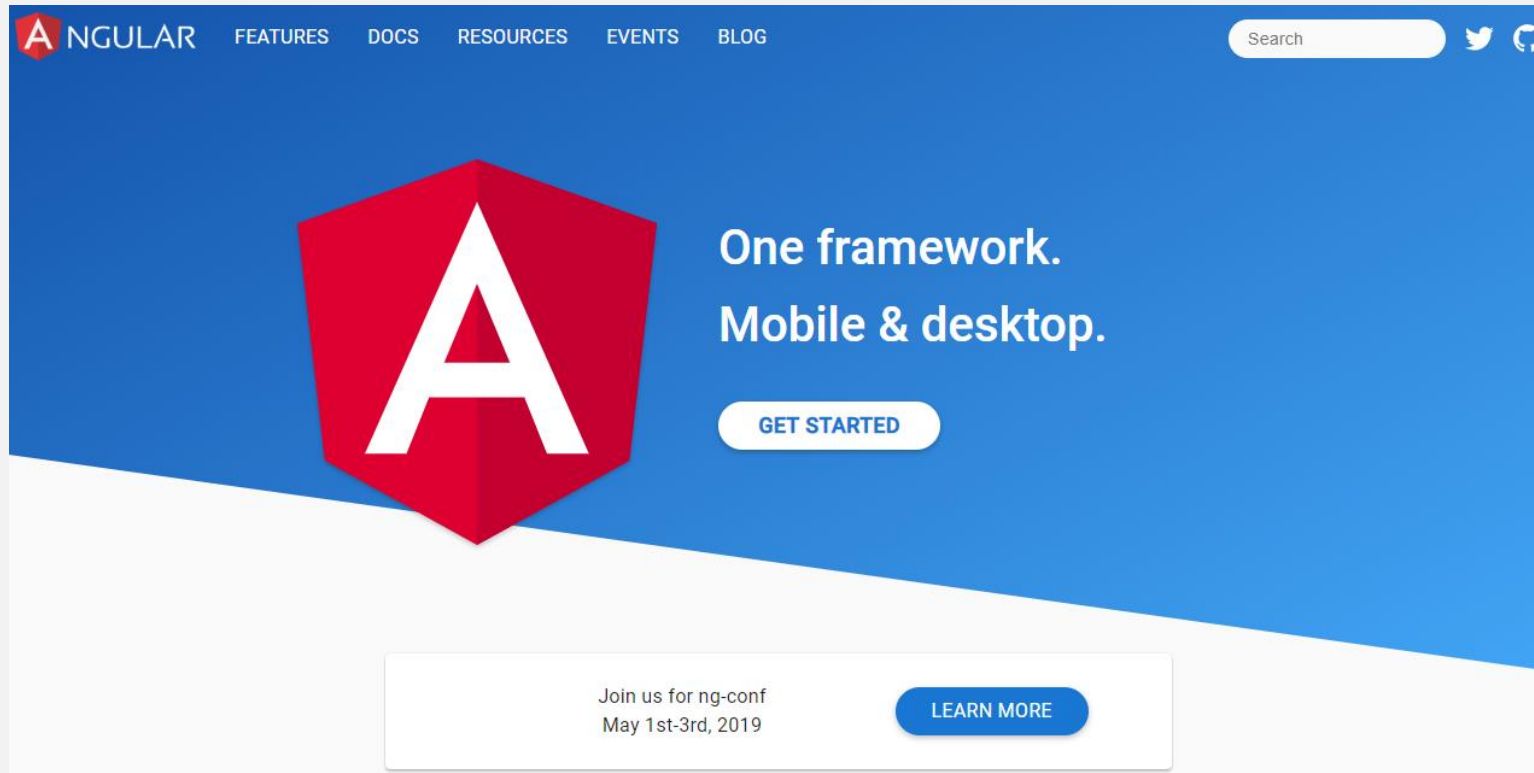
Podemos ainda configurar os ficheiros JavaScript gerados usando um ficheiro de configuração “tsconfig.json”

```
{
  "compilerOptions": {
    "outDir": "./built",
    "allowJs": true,
    "target": "es5"
  },
  "include": [
    "./src/**/*.ts"
  ]
}
```

TypeScript

Demonstração

Introdução ao Angular



Fonte: <https://angular.io/>

Introdução ao Angular

Foi criado pela Google em 2012

Foi construída tendo por base o padrão MVC que conhecemos e obriga os seus projetos a implementá-lo

Foi a provavelmente a primeira JavaScript Frameworks orientada ao desenvolvimento do lado do cliente com grande sucesso

Introdução ao Angular

Angular é uma plataforma e estrutura para criar aplicações do lado do cliente

Angular é atualmente escrito em TypeScript

Introdução ao Angular

Os blocos de construção básicos de uma aplicação Angular são chamados NgModules, que:

- definem views, que são conjuntos de elementos no ecrã que o Angular pode escolher entre e modificar de acordo com a lógica e os dados do programa
- usam serviços, que fornecem funcionalidade específica não diretamente relacionada a visualizações

Introdução ao Angular

NgModules

- Os NgModules de Angular diferem e complementam os módulos JavaScript (ES2015)
- Um NgModule declara um contexto de compilação para um conjunto de componentes e um fluxo de trabalho ou um conjunto estreitamente relacionado de recursos
- Um NgModule pode associar seus componentes ao código relacionado, como serviços, para formar unidades funcionais

Introdução ao Angular

NgModules

- Cada aplicação Angular tem um módulo raiz, chamado convencionalmente AppModule, que fornece o mecanismo de inicialização que inicia o aplicação
- Uma aplicação geralmente contém muitos módulos funcionais
- Como os módulos JavaScript, o NgModules pode importar funcionalidades de outros NgModules e permitir que suas próprias funcionalidades sejam exportadas e usadas por outros NgModules

Introdução ao Angular

AppModule

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { HttpClientModule } from '@angular/common/http';
import { RouterModule } from '@angular/router';

import { AppRoutingModuleModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { PeopleComponent } from './people/people.component';

@NgModule({
  declarations: [
    AppComponent,
    PeopleComponent
  ],
  imports: [
    BrowserModule,
    HttpClientModule,
    FormsModule,
    AppRoutingModuleModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Introdução ao Angular

Componentes

- Cada componente define:
 - uma classe que contém dados e lógica (typescript)
 - um modelo para a estrutura do componente (html)
 - Um modelo de formatação do componente (css)
- Cada aplicação Angular tem pelo menos um componente, o componente raiz que conecta uma hierarquia de componentes ao modelo de objeto de documento de página (DOM).

Introdução ao Angular

Componentes

- O decorador `@Component()` identifica:
 - A classe lógica
 - O template html
 - O estilo do componente em css

```
fabio, 5 months ago | 1 author (fabio)
import { Component } from '@angular/core';

fabio, 5 months ago | 1 author (fabio)
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'tutorial';
}
```

Introdução ao Angular

Componentes

```
fabio, 5 months ago | 1 author (fabio)  
import { Component } from '@angular/core';
```

```
fabio, 5 months ago | 1 author (fabio)  
@Component({  
  selector: 'app-root',  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})  
export class AppComponent {  
  title = 'tutorial';  
}
```

```
import { BrowserModule } from '@angular/platform-browser';  
import { NgModule } from '@angular/core';  
import { FormsModule } from '@angular/forms';  
import { HttpClientModule } from '@angular/common/http';  
import { RouterModule } from '@angular/router';  
  
import { AppRoutingModuleModule } from './app-routing.module';  
import { AppComponent } from './app.component';  
import { PeopleComponent } from './people/people.component';  
  
@NgModule({  
  declarations: [  
    AppComponent,  
    PeopleComponent  
  ],  
  imports: [  
    BrowserModule,  
    HttpClientModule,  
    FormsModule,  
    AppRoutingModuleModule  
  ],  
  providers: [],  
  bootstrap: [AppComponent]  
})  
export class AppModule { }
```

Introdução ao Angular

Templates

- Um modelo combina HTML com marcação angular que pode modificar elementos HTML antes de serem exibidos
- As diretivas de modelo fornecem lógica de programa e a marcação de ligação conecta os dados do aplicação e o DOM
- Todo o processamento dos templates é feito do lado do cliente

Introdução ao Angular

Templates

- Antes de a marcação ser exibida, o Angular avalia as diretivas e resolve a sintaxe de ligação no modelo para modificar os elementos HTML e o DOM, de acordo com os dados e a lógica do programa
- O Angular suporta ligação de dados bidirecional, o que significa que alterações no DOM, como opções do utilizador, também são refletidas nos dados do programa

Introdução ao Angular

Templates

```
You, a few seconds ago | 2 authors (fabio and others)
<!--The content below is only a placeholder and can be replaced.-->
<div style="text-align:center">
  <h1>
    Welcome to {{ title }}!
  </h1>
</div>
<h2>Uma lista de pessoas </h2>
<app-people></app-people>

<router-outlet></router-outlet>
```

Introdução ao Angular

Serviços e Injeção de Dependências

- Para dados ou lógica que não estão associados a uma vista específica e onde é necessário compartilhar entre componentes deve ser criado uma classe de serviço
- Uma definição de classe de serviço é imediatamente precedida pelo decorador `@Injectable ()`
- O decorador fornece os metadados que permitem que outros provedores sejam injetados como dependências em sua classe.

Introdução ao Angular

Serviços e Injeção de Dependências

- A injeção de dependência (DI) permite manter as classes de componentes isoladas e eficientes
- Os serviços podem obter dados de um servidor ou validar a inputs do utilizador

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root',
})
export class HeroService {

  constructor() { }

}
```

Introdução ao Angular

Routing

- O Angular Router NgModule fornece um serviço que permite definir um caminho de navegação entre os diferentes estados da aplicação e exibir hierarquias na aplicação
- O router mapeia caminhos semelhantes a URL para visualizações em vez de páginas
- Quando um utilizador executa uma ação, como clicar num link, que carrega uma nova página no browser, o router intercepta o comportamento do browser e mostra ou oculta as hierarquias de exibição dos componentes em Angular

Introdução ao Angular

Routing

- Se o router determinar que o estado atual do aplicação requer uma funcionalidade específica e o módulo que o define não tiver sido carregado, o router carregará o módulo
- O router interpreta um URL de link de acordo com as regras de navegação e o estado de dados e hierarquias de visualização da aplicação
- O router registra a atividade no histórico do browser, de modo que os botões de retrocesso e de encaminhamento também funcionam

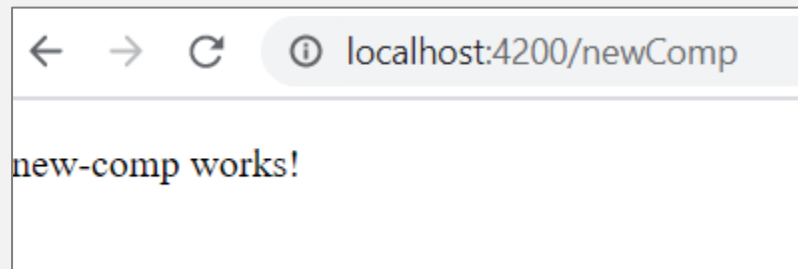
Introdução ao Angular

Routing

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';
import { NewCompComponent } from './new-comp/new-comp.component';

const routes: Routes = [
  {path: 'newComp', component: NewCompComponent}
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```



Introdução ao Angular

Componentes e serviços são simplesmente classes, com decoradores que marcam seu tipo e fornecem metadados que informam ao Angular como usá-los

Os metadados de uma classe de componentes associam a um modelo que define uma visão

Um modelo combina HTML comum com diretivas angulares e marcação de ligação que permitem que o Angular modifique o HTML antes de exibi-lo para exibição

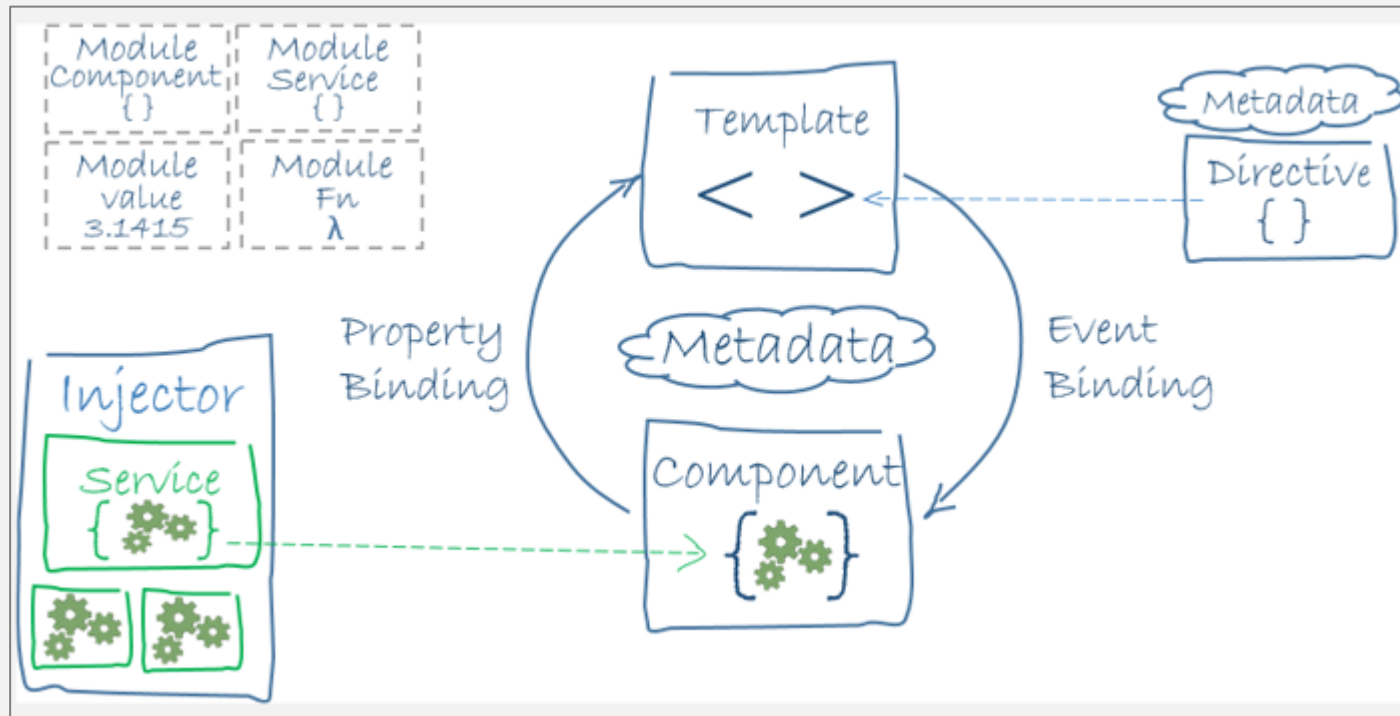
Introdução ao Angular

Os metadados para uma classe de serviço fornecem as informações que o Angular precisa para disponibilizá-los aos componentes por meio da injeção de dependência (DI)

Os componentes de um aplicação geralmente definem várias exibições, organizadas hierarquicamente

Angular fornece o serviço router para ajudá-lo a definir caminhos de navegação entre visualizações. O router oferece recursos sofisticados de navegação no browser

Introdução ao Angular



Fonte: <https://angular.io/generated/images/guide/architecture/overview2.png>

Instalação Angular

Para iniciar o desenvolvimento em Angular é necessário antes demais garantir que se encontram instalados:

- runtime Node.js
- package Manager npm

A plataforma de desenvolvimento de Angular é depois instalada com

- `npm install -g @angular/cli`

Instalação Angular

Podemos iniciar um projeto em Angular criando um projeto inicial com o comando:

- `ng new my-app`

Após o comando devem ser criados

- Um novo espaço de trabalho, com uma pasta raiz chamada my-app
- Um projeto de aplicação esqueleto inicial, chamado my-app (na subpasta src)
- Um projeto de teste (na subpasta e2e)
- Arquivos de configuração relacionados
- O projeto inicial contém uma aplicação de boas-vindas simples, pronta a ser executada.

Instalação Angular

Para correr um projeto em angular podemos usar o comando:

- npm start

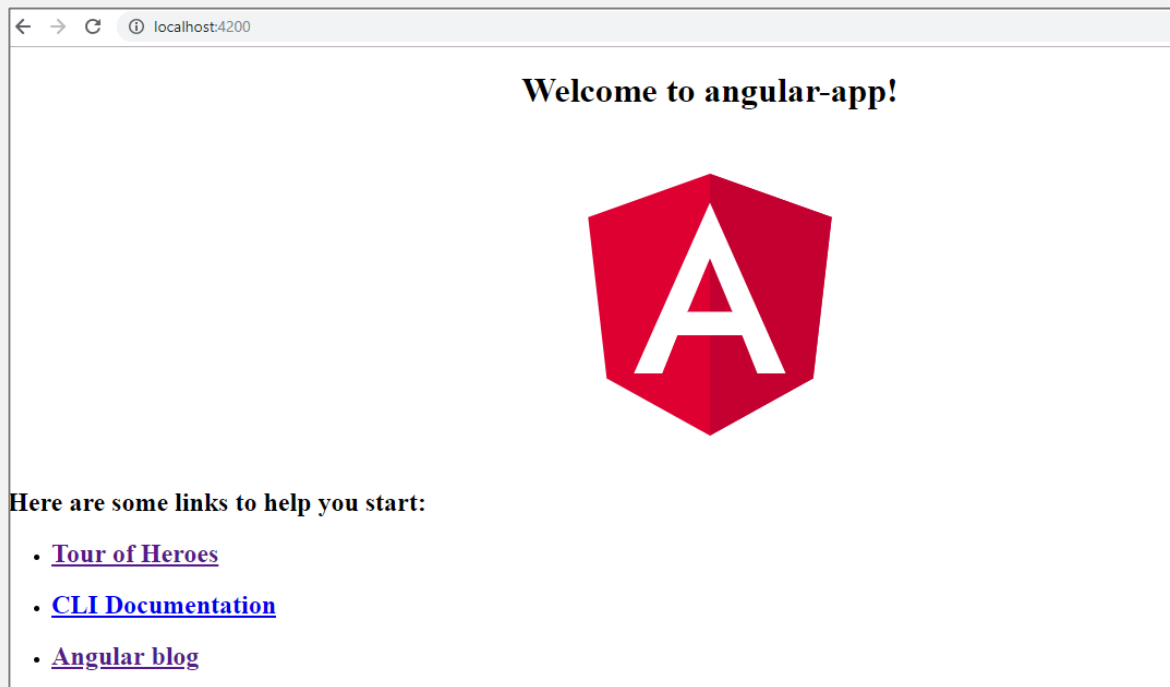
ou o comando:

- ng serve –open

Em qualquer dos casos a aplicação é compilada e depois executada num servidor interno da plataforma Angular

Demo Angular

Aplicação exemplo



Demo Angular

Podemos adicionar um novo componente à aplicação usando apenas o comando

- ng generate component “nome”

É criada automaticamente uma pasta com 4 ficheiros acerca do novo componente

```
✓ new-component  
# new-component.component.css  
<> new-component.component.html  
TS new-component.component.spec.ts  
TS new-component.component.ts
```

Demo Angular

As aplicações em Angular são escritas em TypeScript o que implica a necessidade de compilar cada aplicação para que os ficheiros escritos em TypeScript sejam convertidos em JavaScript

Iremos aprofundar o estudo de Angular nas próximas aulas...

Demo Angular

Demonstração Prática

Referências

Typescript

- <https://www.typescriptlang.org/>

Angular

- <https://angular.io/>
- <https://angular.io/docs>

P.PORTO

Frameworks do lado do cliente

Programação em Ambiente Web